



Software Engineering

A D Patel Institute of
Technology



AY:2025-26
SEM-V, Div: IT1

Practical -1

Aim: An introduction to software engineering & Studying Various phases of Water-Fall Model. For each SDLC phase, identify the objectives and summaries outcomes.

➤ What is Software Engineering?

Software engineering is a systematic and disciplined approach to designing, developing, testing, deploying, and maintaining software. It applies engineering principles to create reliable and efficient software systems that meet user requirements.

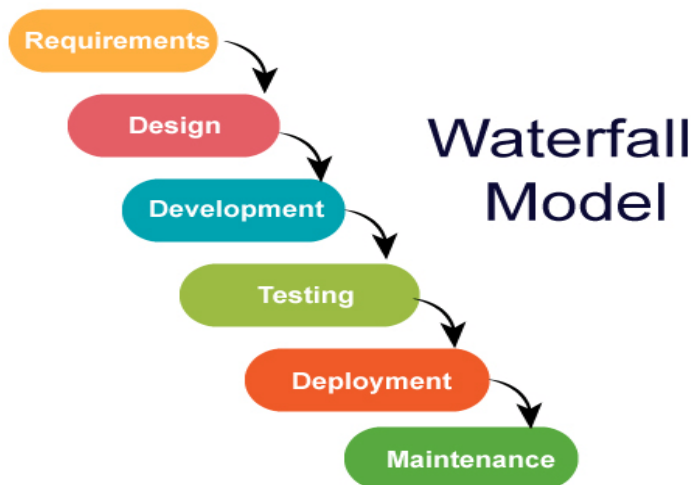
Think of it as the difference between a hobbyist building a wooden shed and a team of architects and construction workers building a skyscraper. While both create a structure, the skyscraper requires rigorous planning, standardized processes, and a focus on reliability, scalability, and safety – that's what software engineering brings to building software.

While coding is a crucial part of software development, it's just one piece of the puzzle. Software engineering encompasses the entire lifecycle of a software product, from the initial idea to its final retirement. Its main goals are to manage **complexity** and ensure quality.

- **Managing Complexity:** Modern software systems can have millions of lines of code and are often developed by large, distributed teams. Software engineering provides the principles and methodologies to manage this complexity, ensuring the project stays on track, on budget, and meets its objectives.
- **Ensuring Quality:** It aims to produce reliable, efficient, and maintainable software that is free of defects. This is achieved through structured processes, rigorous testing, and continuous review.

➤ Waterfall model

The **Waterfall Model** is one of the earliest and most straightforward Software Development Life Cycle (SDLC) models. It follows a linear, sequential approach where each phase must be fully completed before the next one begins, flowing downwards like a waterfall.



1. Requirements Analysis and Specification

- **Objective:** To understand, capture, and document the customer's needs and system requirements in detail. This phase focuses on *what* the system is supposed to do, not *how* it will do it.
- **Outcomes:** The primary outcome is the **Software Requirement Specification (SRS)** document. This comprehensive document acts as a contract between the client and the developers and typically includes:
 - **Functional Requirements:** Specific services the system must provide.
 - **Non-Functional Requirements:** Constraints such as performance, security, and reliability.
 - **User and System Goals.**

2. System Design

- **Objective:** To transform the requirements from the SRS into a blueprint for constructing the software. This phase defines the overall system architecture, components, modules, and data structures.
- **Outcomes:** This phase produces design documents that guide the implementation.



Software Engineering

A D Patel Institute of
Technology



AY:2025-26
SEM-V, Div: IT1

- **High-Level Design (HLD):** Describes the architecture, database design, and relationships between different modules.
- **Low-Level Design (LLD):** Details the functional logic of each module, including data structures and algorithms.

3. Implementation and Unit Testing

- **Objective:** To write the source code for the software based on the design documents. The system is developed in small units or modules, and each unit is tested individually (unit testing) to ensure it works correctly.
- **Outcomes:**
 - **Source Code:** The actual, completable, and executable code for all modules.
 - **Unit Test Cases & Results:** Documentation of tests performed on individual code components to verify their functionality.

4. Integration and System Testing

- **Objective:** To combine the individually developed units and test the entire system as a whole. The goal is to verify that all components work together correctly and that the complete system meets all requirements specified in the SRS.
- **Outcomes:**
 - **A fully integrated and tested software system.**
 - **Test Report:** A summary of all test cases, their results, and a list of defects found and fixed during the testing process.

5. Deployment of System

- **Objective:** To release the tested and verified software product to the customer or make it available in the market. This involves installing the software in the target environment and making it operational for end-users.
- **Outcomes:**
 - **The final, working software** is delivered and installed.
 - **User Manuals and Documentation:** Guides for installation, usage, and support.

6. Maintenance

- **Objective:** To provide post-delivery support. This is the longest phase, involving modifications to the software to correct errors discovered during use, adapt to a changing environment (e.g., a new operating system), or improve performance and functionality.
- **Outcomes:**
 - **Updated versions** of the software (patches or new releases).
 - **Change Request Logs** and updated documentation.
 - **Bug fix reports.**

➤ Conclusion

The Waterfall Model is a foundational concept in software engineering, prized for its **simplicity and structured approach**. Its rigid, phase-by-phase progression makes it easy to manage and is well-suited for projects where the requirements are clear, well-understood, and unlikely to change. However, its greatest strength is also its most significant weakness. The model's **inflexibility** makes it very difficult and costly to go back and make changes to a previous phase once it is completed. This lack of adaptability is a major drawback in today's dynamic environment, where requirements often evolve. The challenges posed by the Waterfall Model's rigidity directly led to the development of more flexible, iterative methodologies like Agile, which are now dominant in the industry.



Software Engineering

A D Patel Institute of
Technology



AY:2025-26
SEM-V, Div: IT1

Practical -2

Aim: Define requirement Gathering and technical requirement specification for selected project.

Dispatching System, this process is like creating a detailed blueprint before starting construction. It ensures that the final product meets the needs of everyone involved, from the dispatcher to the hospital staff.

➤ Requirement Gathering

Requirement Gathering is the process of collecting and documenting the needs and expectations of the project's stakeholders. It's about figuring out exactly what the system should do, who will use it, and what problems it needs to solve.

Key Stakeholders: To gather requirements for this system, you would need to consult with:

- **Dispatchers:** To understand how they currently handle calls, what information they need, and what their pain points are.
- **Ambulance Crew:** To understand what information they need on the go, how they report status, and what kind of mobile interface they require.
- **Hospital Staff:** To determine how they receive patient information, what a seamless admission process looks like, and what data they need from the system.
- **System Administrators:** To understand security, user management, and maintenance needs.
- **End Users (Public):** Though they don't use the system directly, their needs (e.g., quick response times) drive the system's core functionality.

Methods of Gathering Requirements: You can use several methods to collect this information:

- **Interviews:** Sit down with dispatchers and hospital staff to ask about their workflow and challenges.
- **Observation:** Spend time in an existing dispatch center to see how things are done manually.
- **Surveys & Questionnaires:** Send out structured forms to a wider group to get feedback on specific features.

Types of Requirements: The requirements are typically categorized into two main types:

- **Functional Requirements:** These define what the system *must do*.
 - The system **must allow** a dispatcher to log a new incident with caller and patient details.
 - The system **must display** the real-time location and status of all available ambulances on a map.
 - The system **must be able to** send a dispatch alert to an ambulance crew's mobile device.
 - The system **must record** the timestamp of every status change (e.g., "en route," "on scene," "at hospital").
 - The system **must generate** incident reports for each completed call.

 Software Engineering	A D Patel Institute of Technology	 AY:2025-26 SEM-V, Div: IT1
---	--	---

- **Non-Functional Requirements:** These define *how* the system should perform. They are concerned with quality attributes.
 - **Performance:** The system **must respond** to a dispatcher's search query in less than 2 seconds. The ambulance tracking map **must update** every 30 seconds.
 - **Reliability:** The system **must have** an uptime of 99.9% to ensure it is always available during emergencies.
 - **Usability:** The dispatcher interface **must be** intuitive and easy to use, even during high-stress situations.
 - **Security:** The system **must require** secure login credentials for all users. Patient data **must be** encrypted.

2. Technical Requirement Specification

The **Technical Requirement Specification** is a detailed document that translates the high-level, business-oriented requirements into specific technical instructions for developers. It's the "how-to" guide that outlines the technical architecture, technologies, and constraints.

Here is what the technical specification for your system would include:

- **Architecture & Design:**
 - **System Architecture:** The system will be a client-server application with a web-based interface for dispatchers and a native mobile app for ambulance crews.
 - **Database Design:** The database will be a relational database (e.g., PostgreSQL or MySQL) to manage structured data like patient records and incident logs. The schema will be based on the ER diagram.
- **Software & Hardware Requirements:**
 - **Server:** A cloud-based server (e.g., AWS, Azure) to ensure high availability and scalability.
 - **Programming Languages:** Backend: Python (with Django or Flask), Frontend: JavaScript (React.js), Mobile: Kotlin for Android, Swift for iOS.
 - **APIs:** Integration with third-party APIs for mapping and GPS tracking (e.g., Google Maps API).
- **Security & Data Management:**
 - **User Authentication:** Use industry-standard OAuth 2.0 or JWT (JSON Web Tokens) for secure user authentication.
 - **Data Encryption:** All sensitive data, including patient information, must be encrypted both in transit (using HTTPS) and at rest (in the database).
 - **Access Control:** Implement role-based access control (RBAC) to ensure dispatchers, ambulance crew, and hospital staff only have access to the data relevant to their role.
- **Performance & Scalability:**
 - **Response Time:** API calls must have a response time of less than 500 milliseconds.
 - **Scalability:** The system must be able to handle up to 50 concurrent dispatchers and 500 active ambulance crews without a significant degradation in performance.



Software Engineering

A D Patel Institute of
Technology



AY:2025-26
SEM-V, Div: IT1

- **Deployment & Maintenance:**

- **Deployment:** Use containerization technologies like Docker and Kubernetes for easy deployment and management.
- **Monitoring:** Implement monitoring tools (e.g., Prometheus, Grafana) to track system performance and send alerts.

➤ **Conclusion**

The Dispatching System requirements outlined above provide a comprehensive blueprint for development. Through stakeholder-driven requirement gathering, we ensure the system is user-centered and addresses real operational challenges. The technical specifications define a scalable, secure, and high-performance architecture that will support critical emergency response functions.

This structured approach minimizes risks, enhances usability, and ensures that the final system not only meets current stakeholder needs but can also adapt to future demands.

Practical -3

Aim: Development of DFD and E-R diagram for the software domain problem.

➤ E- Diagram for Ambulance Dispatching System

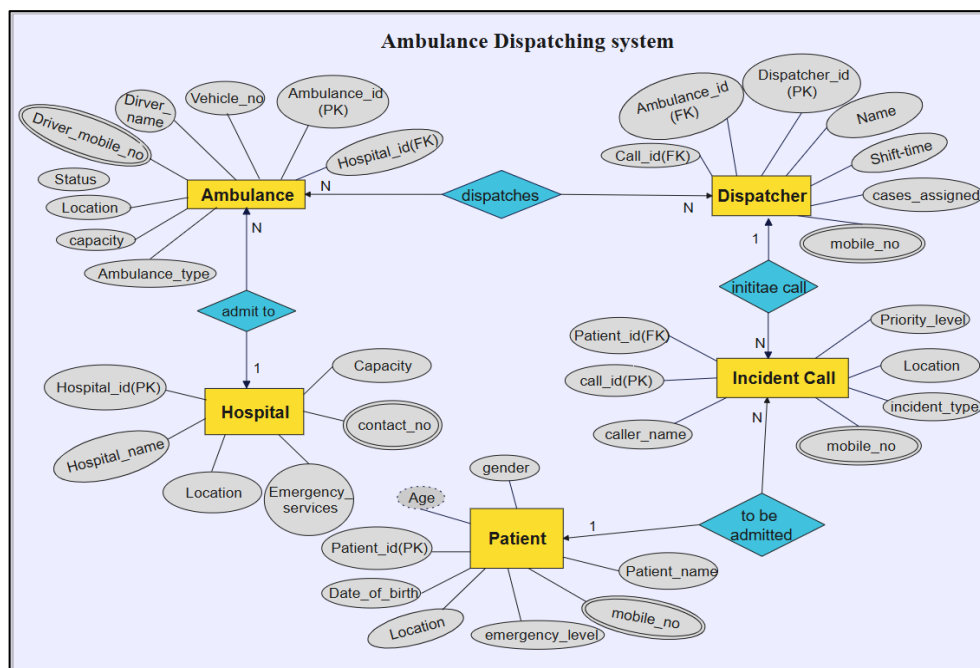
An **E-R Diagram** models the system's data at a conceptual level, identifying key entities, attributes, and relationships. It provides the basis for the database schema.

- Key Entities & Attributes

- **Dispatcher:** Dispatcher_ID, Name, Contact, Role.
- **Ambulance:** Ambulance_ID, Location, Status (available, en route, at scene, at hospital).
- **Incident:** Incident_ID, Caller_Name, Caller_Contact, Location, Timestamp.
- **Patient:** Patient_ID, Name, Age, Condition, Incident_ID (FK).
- **Hospital:** Hospital_ID, Name, Address, Capacity.
- **Ambulance Crew:** Crew_ID, Name, Role, Ambulance_ID (FK).

- Relationships

- **Dispatcher logs** → **Incident** (One-to-Many).
- **Incident involves** → **Patient** (One-to-Many).
- **Incident assigned** → **Ambulance** (Many-to-One).
- **Ambulance operated by** → **Crew** (One-to-Many).
- **Ambulance delivers** → **Patient to Hospital** (Many-to-One).





Software Engineering

A D Patel Institute of
Technology



AY:2025-26
SEM-V, Div: IT1

➤ Data Flow Diagram (DFD)

A **Data Flow Diagram (DFD)** illustrates how data moves through the system, showing inputs, processes, and outputs. It is useful for visualizing the flow of information between different system components and stakeholders.

- Context Level (Level 0 DFD)

- **Entities:**

- Caller (Public) → provides incident details.
- Dispatcher → records and manages incidents.
- Ambulance Crew → receives alerts and updates status.
- Hospital Staff → receives patient information.

- **System:** The Dispatching System acts as the central hub, receiving data from external entities and transmitting processed information back.

- Level 1 DFD (Decomposition of Dispatching System)

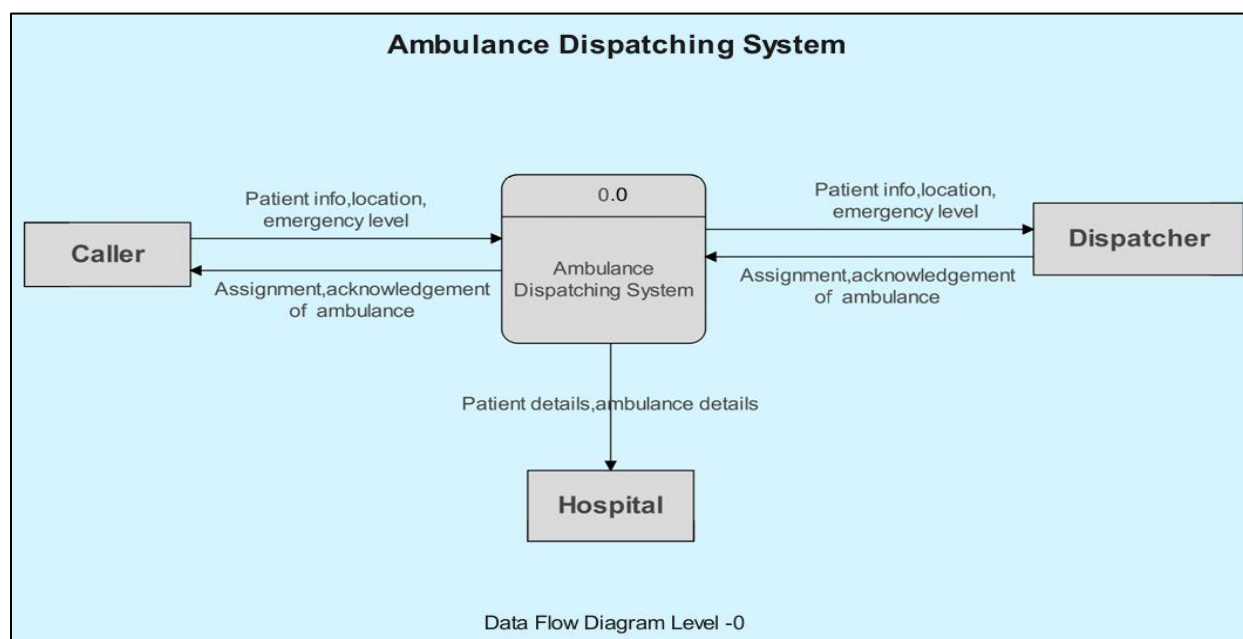
- **Processes:**

- *Incident Logging:* Dispatcher records caller and patient details.
- *Resource Allocation:* System assigns available ambulances.
- *Notification & Tracking:* System sends alerts to ambulance crew and updates real-time location.
- *Reporting:* System generates reports and transmits patient details to hospitals.

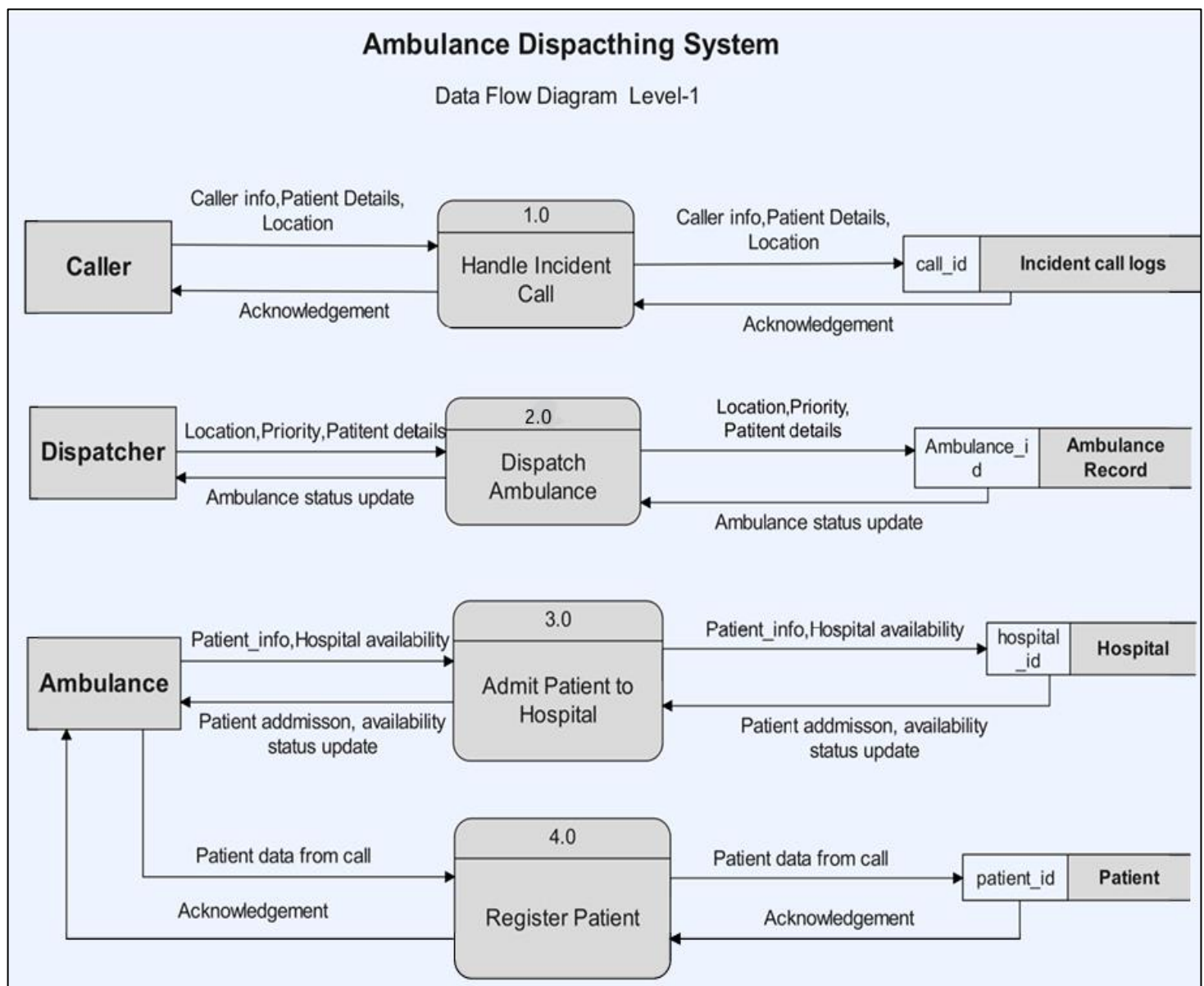
- **Data Stores:**

- Incident Database.
- Ambulance Availability Database.
- Patient Information Database.

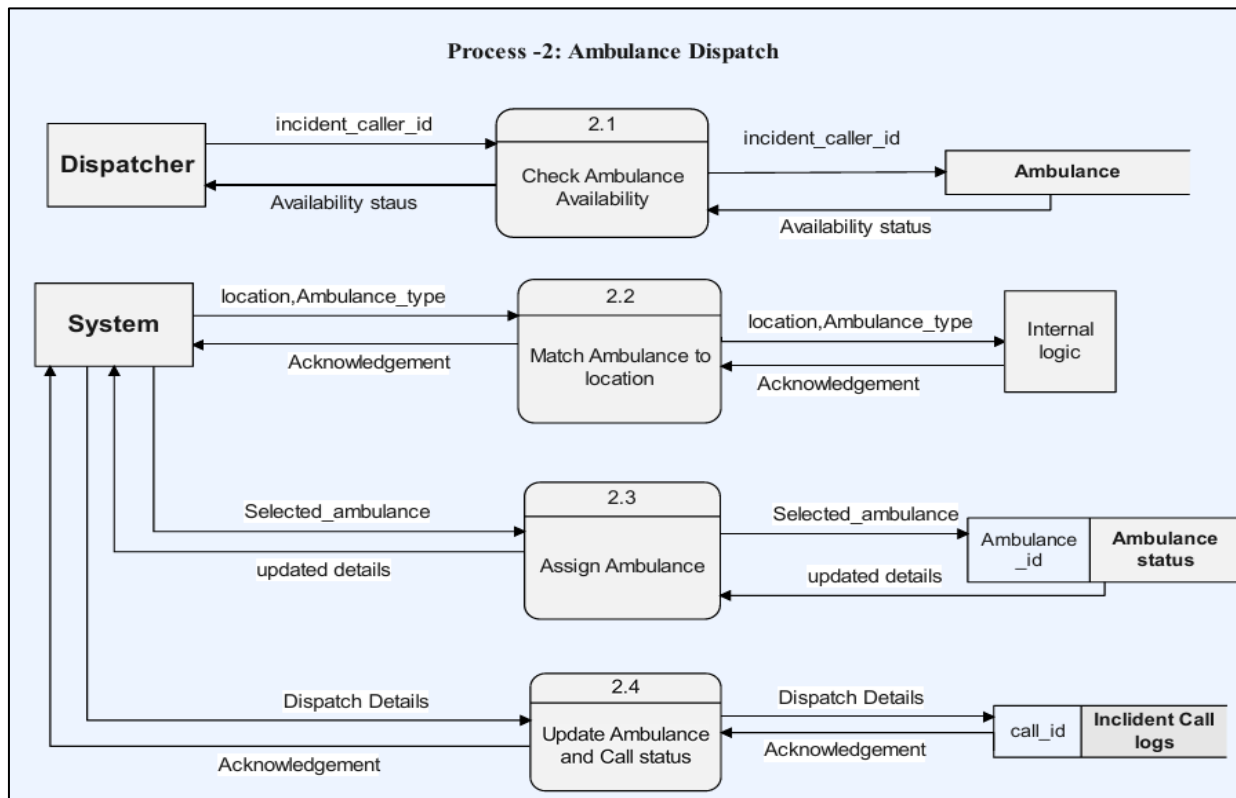
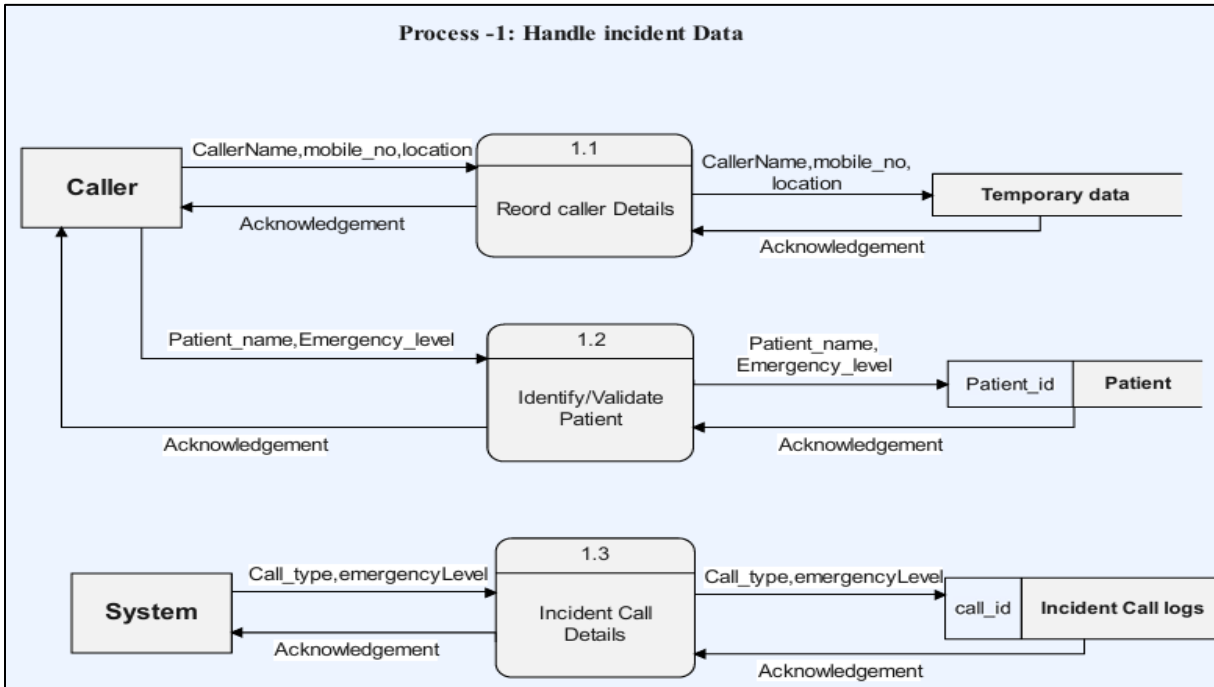
➤ Data flow diagram Level 0



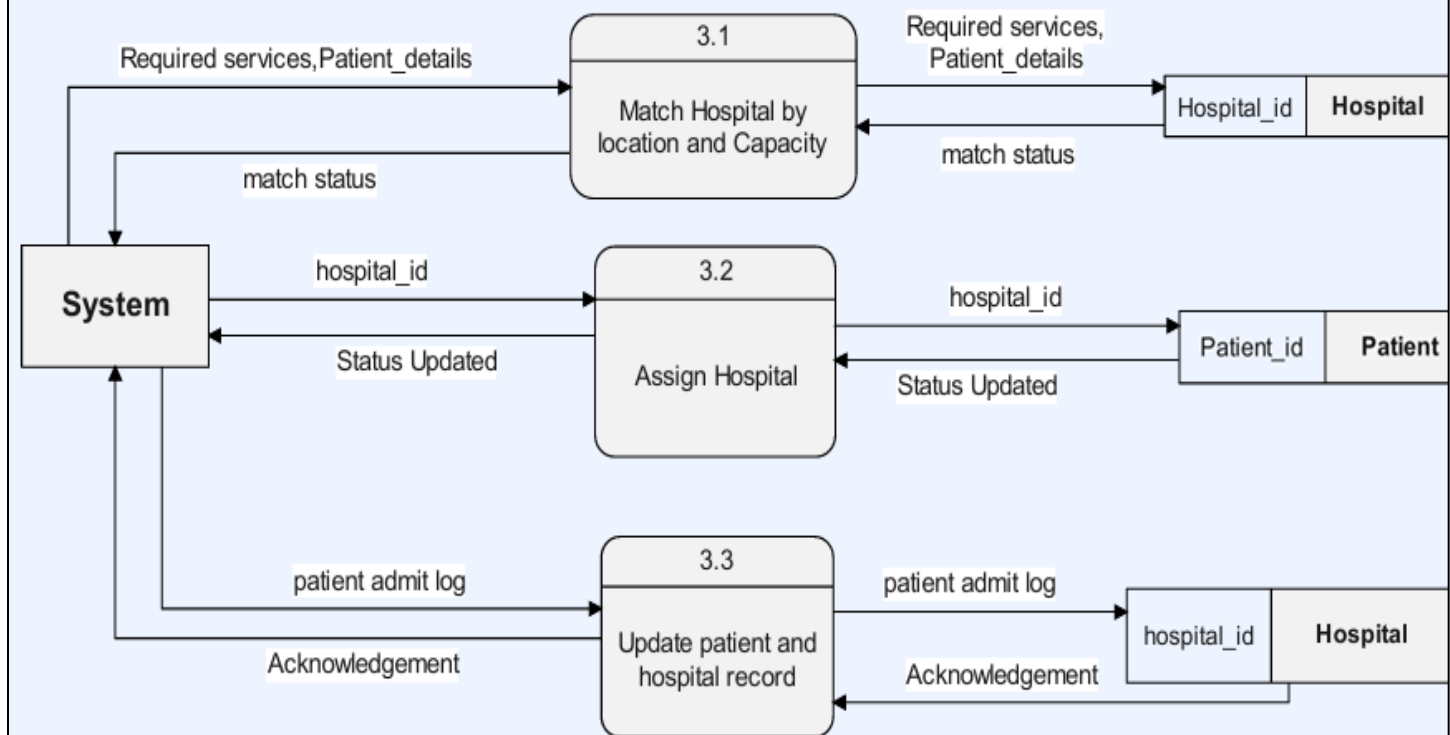
➤ Data flow diagram Level 1



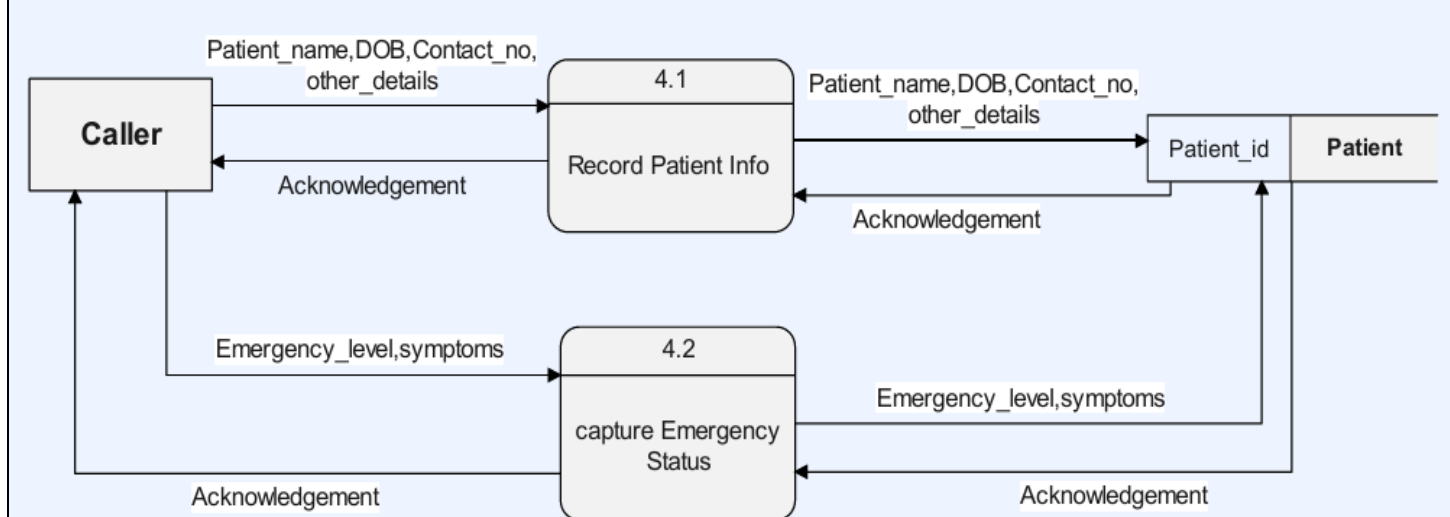
➤ Data flow diagram Level 2



Process -3: Admit Patient to Hospital



Process -4: Register Patient





Software Engineering

A D Patel Institute of
Technology



AY:2025-26
SEM-V, Div: IT1

➤ Conclusion

The DFD provides a clear picture of how information flows across the Dispatching System, highlighting processes, data stores, and interactions with external entities. The E-R Diagram complements this by defining the structural relationships between system entities, ensuring the database is robust, scalable, and consistent.

Together, these models act as a blueprint for system design and implementation, minimizing ambiguity, supporting clear communication among stakeholders, and ensuring that both data management and process flows align with the operational goals of the dispatching system.

Practical -4

Aim: Draw Use case and Activity Diagrams for the project definition.

➤ Use Case Diagram

A Use Case Diagram represents the interactions between system actors (users or external systems) and the system itself. It helps to capture functional requirements from the end-user perspective.

➤ Key Actors

1. Dispatcher – logs incidents, assigns ambulances, monitors operations.
2. Ambulance Crew – receives alerts, updates status, transports patients.
3. Hospital Staff – receives patient data for admission.
4. System Administrator – manages users, roles, and security.
5. Caller (Public) – initiates incident reporting (indirect actor).

➤ Main Use Cases

- Log Incident: Dispatcher records caller and patient details.
- Assign Ambulance: System suggests and dispatcher confirms ambulance allocation.
- Track Ambulance: Dispatcher monitors ambulance status/location.
- Send Dispatch Alert: System notifies ambulance crew.
- Update Status: Ambulance crew reports progress (e.g., en route, at scene).
- Transfer Patient Data: System forwards information to hospital staff.
- Generate Reports: Dispatcher or admin reviews incident reports.
- Manage Users: Admin sets up roles, permissions, and security policies.

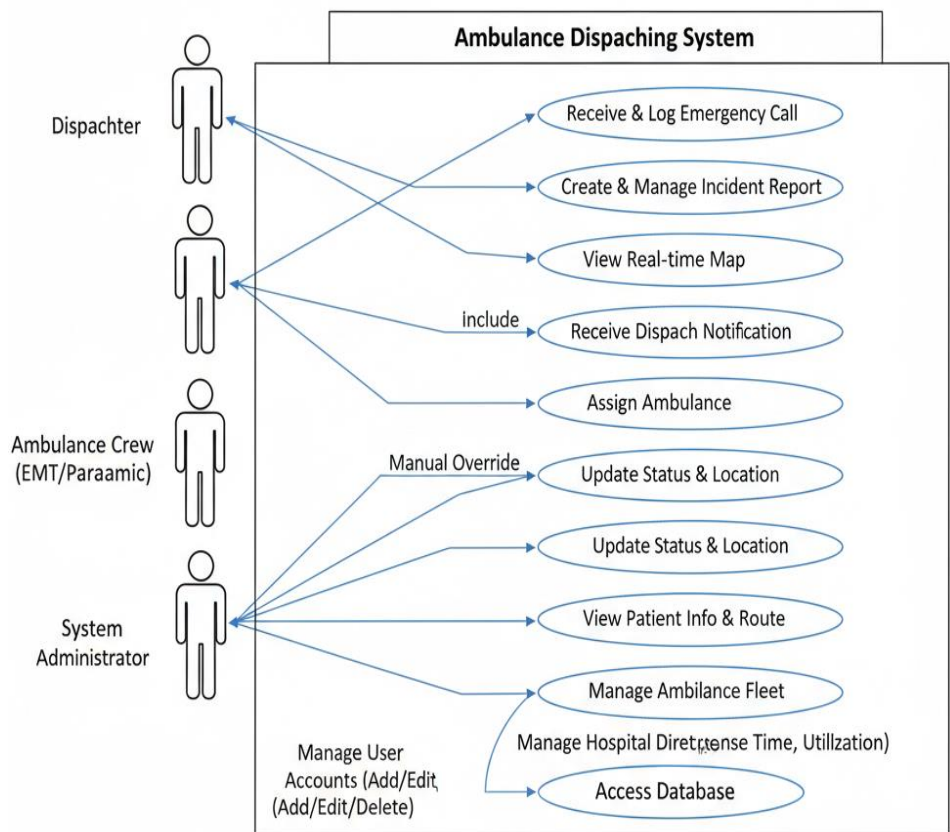


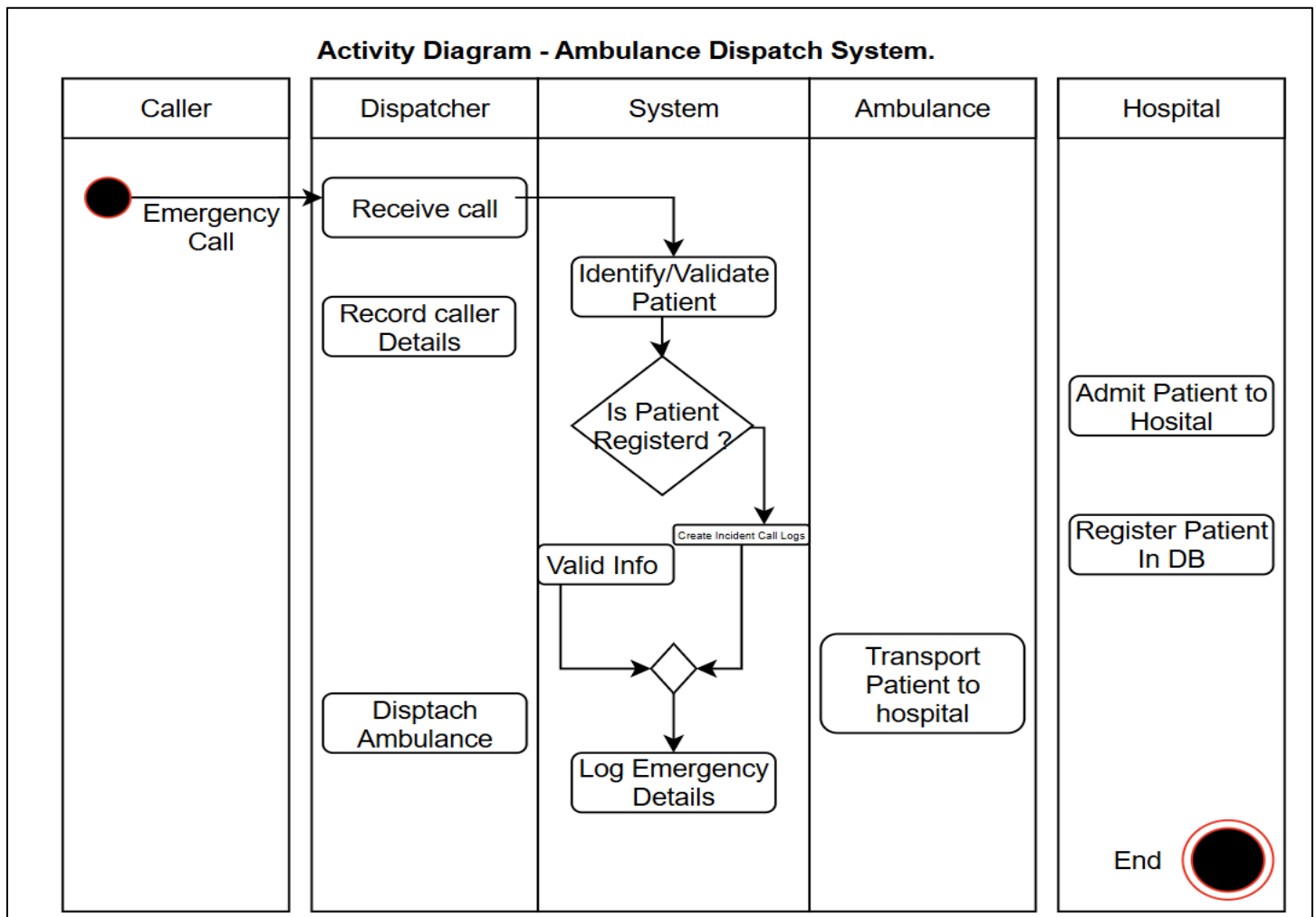
Figure 1: Use Case Diagram for the Ambulance Dispatching System

➤ Activity Diagram

An **Activity Diagram** illustrates the **workflow of activities** in the system, showing how different processes are carried out step by step.

➤ Example Workflow: Emergency Dispatch Process

1. **Caller Reports Incident** → call received.
2. **Dispatcher Logs Details** → enters incident, patient, and location info.
3. **System Checks Available Ambulances** → retrieves data from ambulance database.
4. **Dispatcher Assigns Ambulance** → selects and confirms allocation.
5. **System Sends Alert** → ambulance crew receives details on mobile app.
6. **Ambulance Crew Updates Status** → marks “en route,” “on scene,” “transporting.”
7. **Hospital Staff Receives Patient Info** → prepares for admission.
8. **Incident Closed & Report Generated** → stored in database for records.





Software Engineering

A D Patel Institute of
Technology



AY:2025-26
SEM-V, Div: IT1

➤ Conclusion

The Use Case and Activity Diagrams together provide a comprehensive understanding of the Dispatching System. The Use Case Diagram highlights the functional scope by showing how different actors—dispatchers, ambulance crews, hospital staff, administrators, and callers—interact with the system to achieve their goals. The Activity Diagram complements this by mapping out the workflow step by step, from the moment an incident is reported to the successful admission of a patient. By combining these two perspectives, the project gains both a high-level view of system functionality and a detailed flow of operations, ensuring that no critical process is overlooked. These diagrams not only improve communication between stakeholders and developers but also serve as a foundation for efficient design, implementation, and validation of the system. Ultimately, they help guarantee that the dispatching system will be user-centered, reliable, and aligned with real-world emergency response needs.



Software Engineering

A D Patel Institute of
Technology



AY:2025-26
SEM-V, Div: IT1

Practical -5

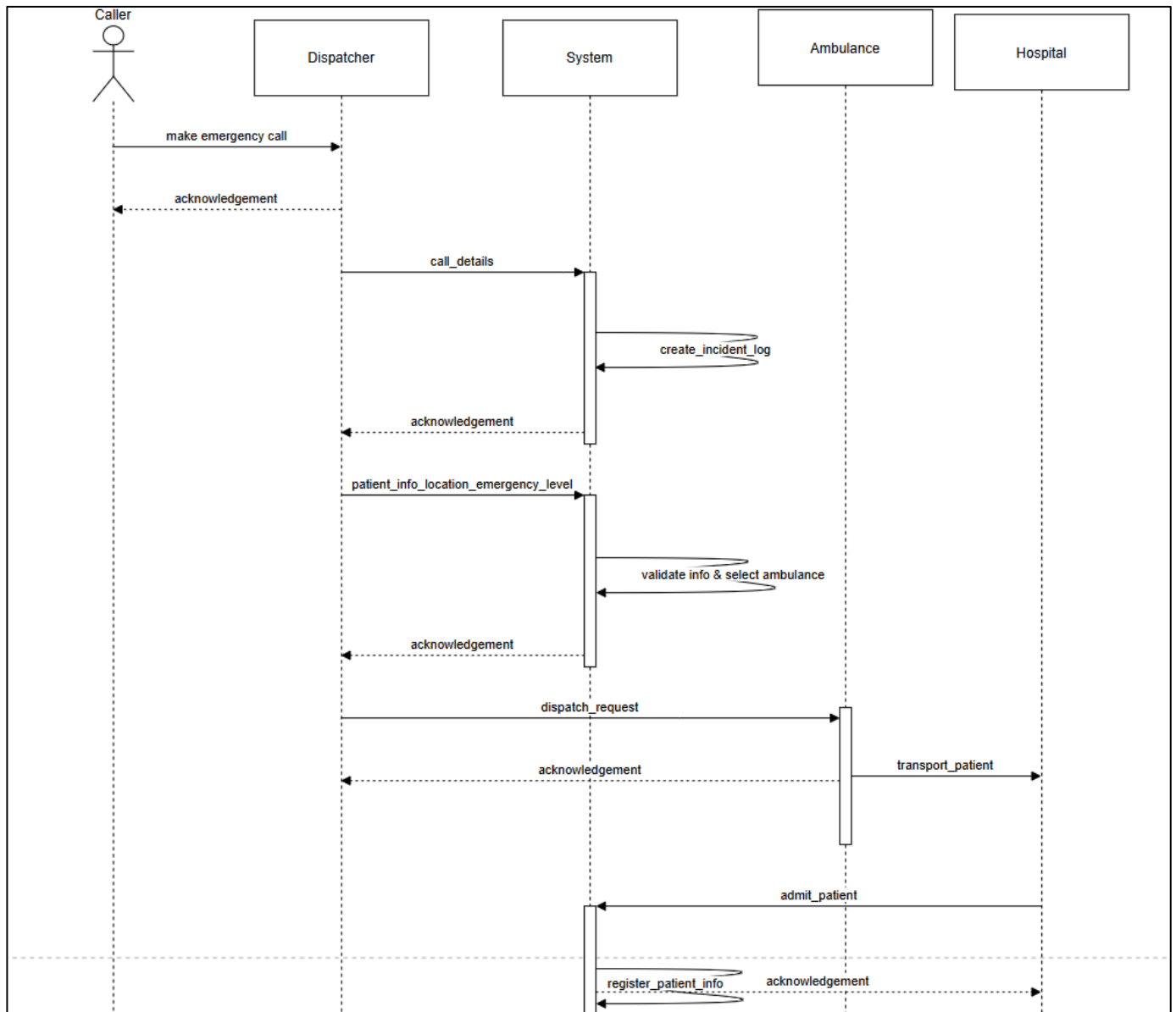
Aim: Draw the Sequence Diagram for the project definition.

A Sequence Diagram is a UML behavioral diagram that models the dynamic interaction between objects in a system, showing how they communicate over time. It emphasizes the order of messages exchanged between actors and system components during a specific process.

In the context of the Dispatching System, the Sequence Diagram helps us visualize the flow of communication that occurs when an emergency is reported, an ambulance is dispatched, and the hospital is updated. It captures the real-time interactions that are critical in ensuring timely emergency response.

➤ Sequence Diagram Description

- Main Objects (Participants)
 - Caller (Public) – initiates the request.
 - Dispatcher – logs the incident and assigns resources.
 - Dispatching System – core system that manages data, alerts, and workflows.
 - Ambulance Crew – receives dispatch notification and updates status.
 - Hospital Staff – receives patient information for preparation and admission.
- Flow of Interaction
 - Caller → Dispatcher: Reports emergency incident.
 - Dispatcher → System: Logs incident details (patient info, location).
 - System → Dispatcher: Confirms incident recorded.
 - System → Ambulance Crew: Sends dispatch alert.
 - Ambulance Crew → System: Updates status (“en route,” “on scene,” etc.).
 - System → Hospital Staff: Sends patient details and estimated arrival time.
 - Hospital Staff → System: Acknowledges readiness to receive patient.
 - System → Dispatcher: Provides final incident report.



➤ Conclusion

The Sequence Diagram provides a clear visualization of how different actors in the Dispatching System interact in chronological order to handle an emergency efficiently. It demonstrates the time-ordered flow of messages from the moment an incident is reported to the hospital's final confirmation of patient admission.



Software Engineering

A D Patel Institute of
Technology



AY:2025-26
SEM-V, Div: IT1

Practical -6

Aim: Development of State Transition Diagram for Software project definition.

➤ State Transition Diagram

A State Transition Diagram (STD) is a behavioral model in UML that represents how an object or system moves from one state to another based on internal or external events. It is particularly useful for systems where the flow of execution depends on the status of processes or event triggers.

In an emergency dispatching system, the lifecycle of an incident or an ambulance assignment can be captured effectively using a State Transition Diagram. It highlights all possible states, events that cause state changes, and the final outcome of the process.

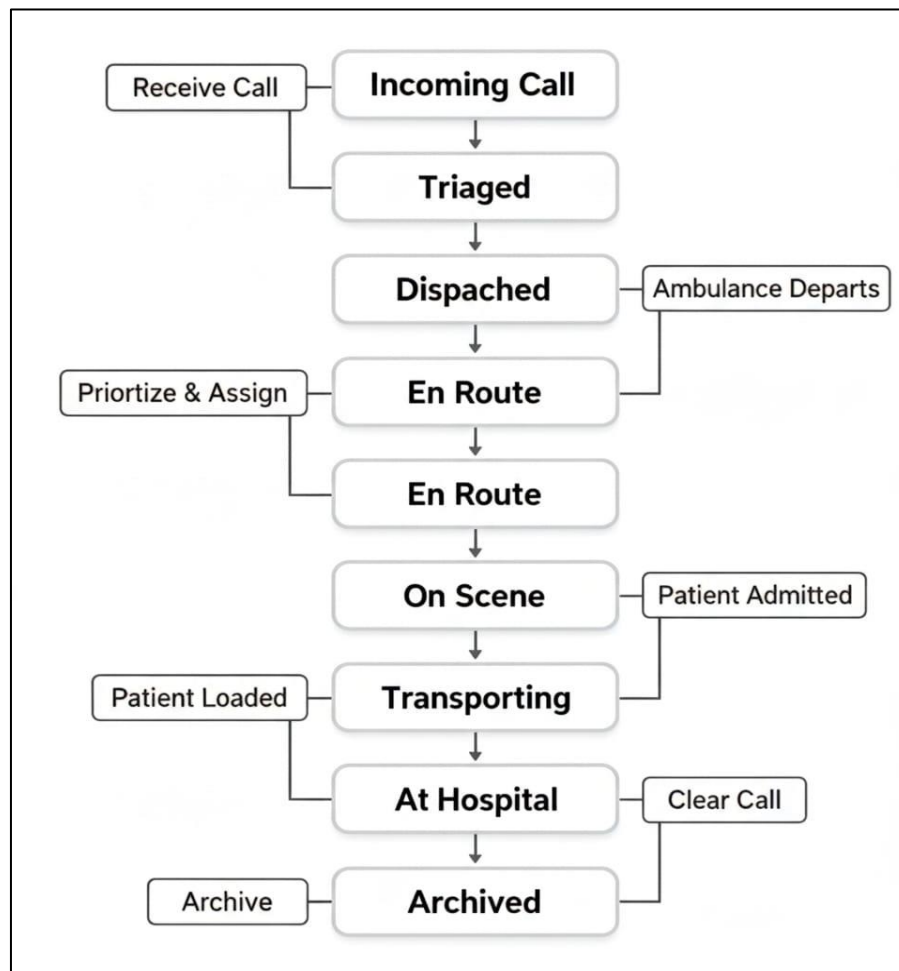
➤ Application to Dispatching System

In this project, the Incident Lifecycle is modeled: from the moment an incident is logged to the point where it is resolved and closed.

Key States and Transitions

1. Idle → Initial state (no active incident).
2. Incident Logged → A dispatcher records caller and patient details.
 - Trigger: Dispatcher submits incident details.
3. Ambulance Assigned → System allocates an available ambulance.
 - Trigger: Dispatcher/system selects ambulance.
4. En Route → Ambulance crew acknowledges and starts traveling.
 - Trigger: Ambulance updates status to “en route.”
5. On Scene → Ambulance arrives at incident location.
 - Trigger: Crew updates status to “on scene.”
6. Transporting Patient → Patient is picked up and transported to hospital.
 - Trigger: Crew updates status to “transporting.”
7. At Hospital → Patient arrives and is handed over.
 - Trigger: Hospital acknowledges patient admission.
8. Incident Closed → Final report generated, incident marked completed.
 - Trigger: Dispatcher/system closes the case.

➤ **Sequence diagram for Ambulance Dispatching System**



➤ **Conclusion**

The State Transition Diagram for the Dispatching System provides a structured view of how an incident progresses through different stages, from logging to closure. It highlights the sequence of states, the events that trigger transitions, and the end conditions of the process.

In essence, the STD ensures that the dispatching system is designed with a **clear lifecycle model**, making it reliable, predictable, and well-suited to handle real-world emergency operations.



Software Engineering

A D Patel Institute of
Technology



AY:2025-26
SEM-V, Div: IT1

Practical -7

Aim: Design Class & Object Diagrams for software domain problem.

➤ Class Diagram

A Class Diagram is a UML structural diagram that represents the static view of a system. It defines the classes, their attributes, operations (methods), and the relationships among them. It serves as the blueprint of the system's data model and functionality.

An Object Diagram is an instance-level snapshot of the class diagram. It shows specific objects (instances of classes) and their relationships at a particular moment in time. It helps validate the class design and demonstrates how the system will look in operation.

➤ Class Diagram for Dispatching System

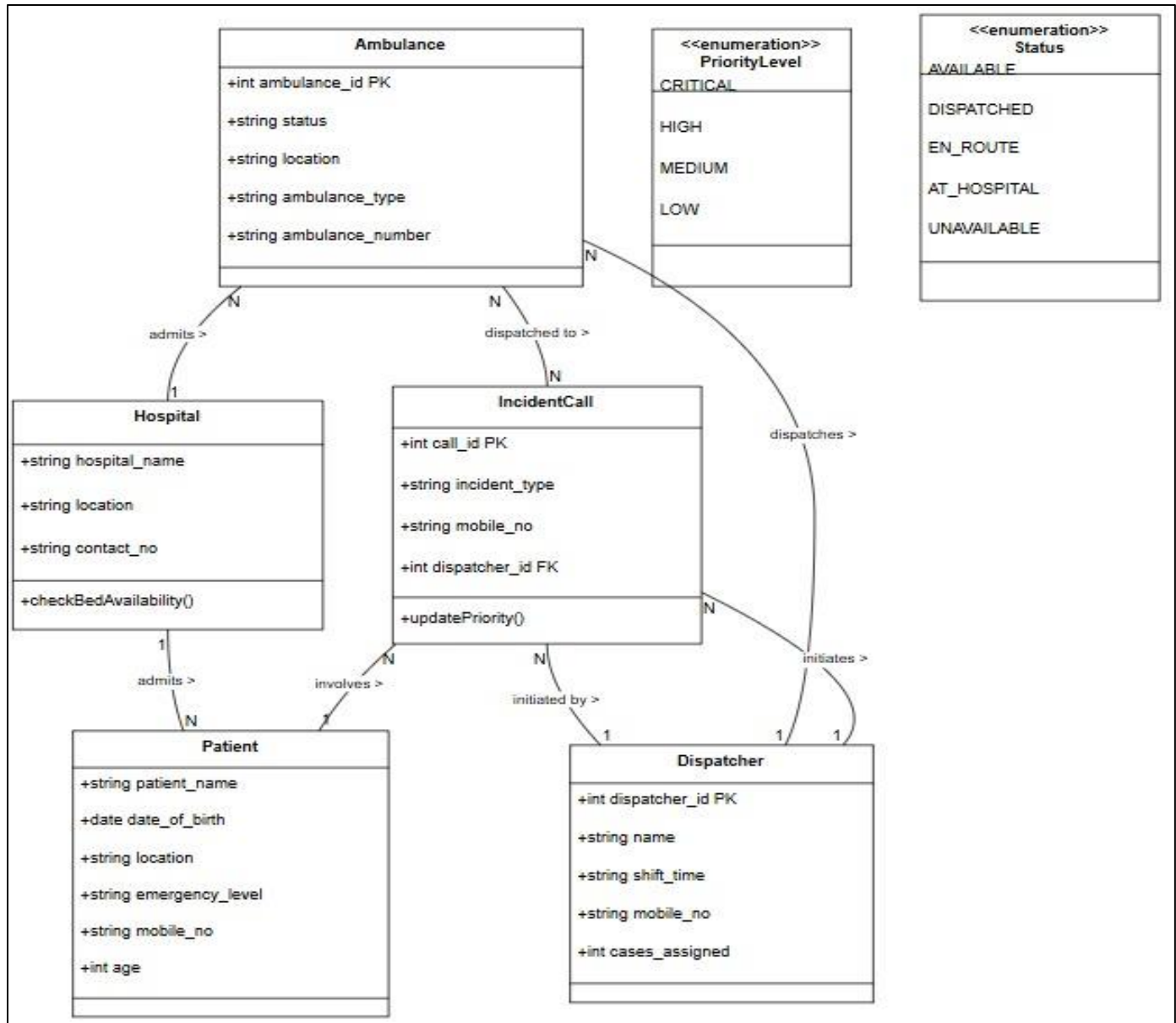
Main Classes and Attributes

1. Dispatcher
 - Attributes: Dispatcher_ID, Name, Contact, Role
 - Methods: logIncident(), assignAmbulance(), generateReport()
2. Incident
 - Attributes: Incident_ID, Location, Timestamp, Status
 - Methods: updateStatus(), closeIncident()
3. Patient
 - Attributes: Patient_ID, Name, Age, Condition
 - Methods: getDetails()
4. Ambulance
 - Attributes: Ambulance_ID, Location, Status
 - Methods: updateLocation(), updateStatus()
5. AmbulanceCrew
 - Attributes: Crew_ID, Name, Role
 - Methods: acknowledgeDispatch(), updateStatus()
6. Hospital
 - Attributes: Hospital_ID, Name, Address, Capacity
 - Methods: receivePatient(), confirmAdmission()
7. SystemAdministrator
 - Attributes: Admin_ID, Name
 - Methods: manageUsers(), manageAccess()

➤ Relationships

- Dispatcher logs → Incident (1..*).
- Incident involves → Patient (1..*).
- Incident assigned to → Ambulance (1..1).
- Ambulance operated by → AmbulanceCrew (1..*).
- Patient admitted to → Hospital (1..1).
- Administrator manages → System Users (1..*).

➤ **Class Diagram**



➤ **Conclusion**

The Class Diagram provides a static structural view of the Dispatching System by defining classes, attributes, methods, and relationships that form the system's foundation. The Object Diagram complements this by showing a real-world snapshot of class instances during an actual emergency scenario.



Software Engineering

A D Patel Institute of
Technology



AY:2025-26
SEM-V, Div: IT1

Practical -8

Aim: Prepare a data dictionary for Software project definition.

➤ Data Dictionary for Ambulance Dispatching System

This data dictionary defines all entities, attributes, data types, keys, and descriptions used in the Ambulance Dispatching System. It is based on the ER diagram and relationships among system components.

1. Entity: Ambulance

Attribute	Data Type	Key	Description
Ambulance_id	Integer	Primary Key	Unique identifier for each ambulance
Vehicle_no	Varchar		Vehicle number of the ambulance
Driver_name	Varchar		Name of the ambulance driver
Driver_mobile_no	Varchar		Contact number of the driver
Status	Varchar		Current status (Available, On-duty)
Location	Varchar		Current location of the ambulance
Capacity	Integer		Maximum number of patients it can hold
Ambulance_type	Varchar		Type of ambulance (ICU, Basic, Advanced)
Hospital_id	Integer	Foreign Key	Associated hospital

2. Entity: Dispatcher

Attribute	Data Type	Key	Description
Dispatcher_id	Integer	Primary Key	Unique identifier for dispatcher
Name	Varchar		Full name of the dispatcher
Shift time	Varchar		Working shift timing
Cases assigned	Integer		Number of cases assigned
Mobile no	Varchar		Dispatcher's contact number

3. Entity: Hospital

Attribute	Data Type	Key	Description
Hospital_id	Integer	Primary Key	Unique identifier for hospital
Hospital_name	Varchar		Name of the hospital
Location	Varchar		Physical location of hospital
Capacity	Integer		Maximum number of patients it can admit
Contact no	Varchar		Hospital contact number
Emergency_services	Varchar		Available emergency services

4. Entity: Patient

Attribute	Data Type	Key	Description
Patient_id	Integer	Primary Key	Unique identifier for patient
Patient_name	Varchar		Full name of patient



Software Engineering

A D Patel Institute of
Technology



AY:2025-26
SEM-V, Div: IT1

Gender	Varchar		Gender of the patient
Age	Integer		Age of the patient
Date of birth	Date		Patient's date of birth
Location	Varchar		Patient's current location
Emergency_level	Varchar		Level of urgency (High, Medium, Low)
Mobile_no	Varchar		Patient or guardian contact number

5. Entity: Incident Call

Attribute	Data Type	Key	Description
Call_id	Integer	Primary Key	Unique identifier for each incident call
Patient_id	Integer	Foreign Key	Patient involved in the call
Caller_name	Varchar		Name of the person who made the call
Priority_level	Varchar		Urgency level of the incident
Location	Varchar		Location where incident occurred
Incident_type	Varchar		Type of incident (Accident, Medical, etc.)
Mobile_no	Varchar		Caller's contact number

6. Relationships

Relationship	Type	Attributes	Description
Ambulance – Dispatcher	Many-to-Many	Ambulance_id (FK), Call_id (FK)	Dispatcher assigns ambulances to incident calls
Dispatcher – Incident Call	One-to-Many	Dispatcher_id (PK), Call_id (FK)	Dispatcher initiates incident calls
Incident Call – Patient	Many-to-One	Patient_id (FK)	Incident involves a patient
Ambulance – Hospital	Many-to-One	Hospital_id (FK)	Ambulance delivers patient to a hospital

➤ Conclusion

This data dictionary provides a comprehensive reference for all entities, attributes, and relationships in the Ambulance Dispatching System. It ensures consistent naming, data integrity, and clarity during database design and software development. Developers and stakeholders can use this as a blueprint for building and testing the system.



Software Engineering

A D Patel Institute of
Technology



AY:2025-26
SEM-V, Div: IT1

Practical -9

Aim: Draw the Deployment Diagram for the project definition.

➤ Deployment Diagram for Ambulance Dispatching System

A Deployment Diagram is a type of UML diagram that shows the physical architecture of a system. It illustrates how software components are mapped onto hardware nodes and how these nodes communicate with each other.

In practical terms, it represents:

- Servers, clients, and devices used in the system.
- Connections and communication paths between them.
- Where each software component resides during runtime.

For an Ambulance Dispatching System, the deployment diagram ensures the system's reliability, scalability, and real-time responsiveness.

➤ Deployment Diagram Description

1. Dispatcher Workstation (Client Node)

- Hardware: Desktop or Laptop
- Software: Web browser / Dispatcher interface
- Function: Allows dispatcher to log incidents, assign ambulances, monitor status, and generate reports

2. Ambulance Mobile Device (Client Node)

- Hardware: Mobile device (Android/iOS)
- Software: Native mobile app
- Function: Receives dispatch alerts, updates status, and tracks real-time location

3. Hospital Workstation (Client Node)

- Hardware: Desktop/Laptop
- Software: Web-based hospital portal
- Function: Receives patient details and confirms admission

4. Application Server (Node)

- Hardware: Cloud-based server (AWS/Azure)
- Software: Backend services (Python/Django or Flask), APIs
- Function: Handles business logic, dispatch assignment, and communication between clients

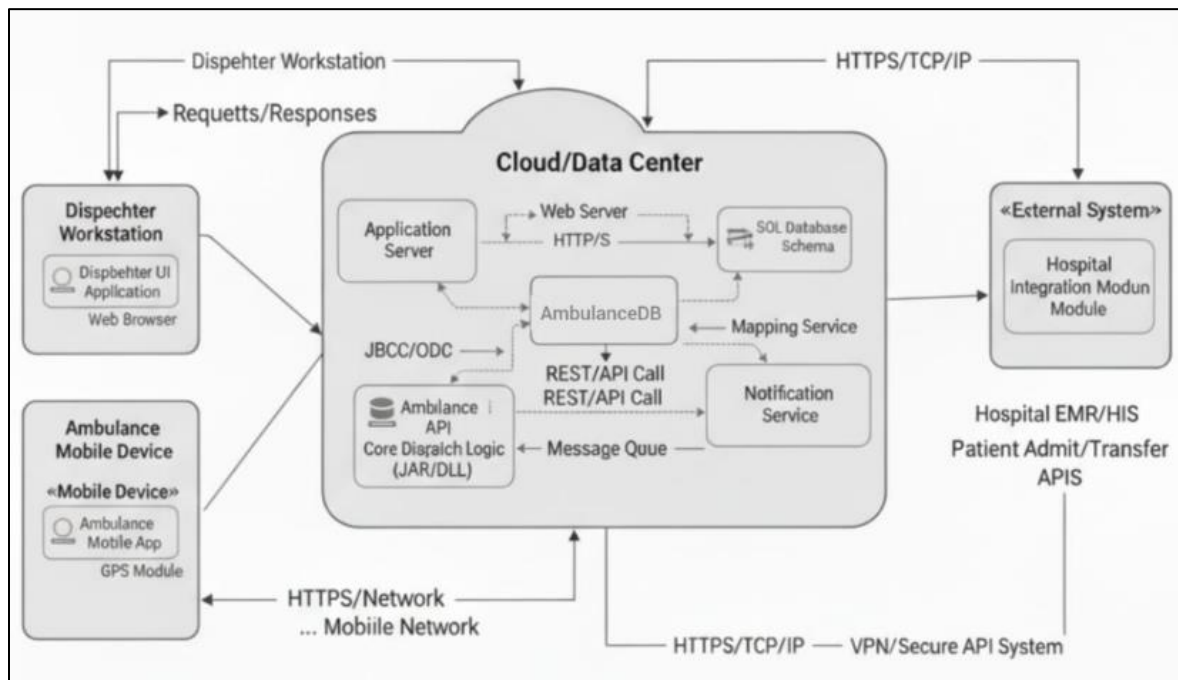
5. Database Server (Node)

- Hardware: Cloud-based or on-premise database server
- Software: PostgreSQL/MySQL
- Function: Stores incident logs, patient data, ambulance and crew details

➤ Communication

- **Dispatcher Workstation ↔ Application Server:** HTTPS requests to log incidents and assign ambulances
- **Ambulance Mobile Device ↔ Application Server:** Real-time updates via API calls or WebSocket
- **Hospital Workstation ↔ Application Server:** Receives patient information and updates admission status
- **Application Server ↔ Database Server:** Queries and updates data

➤ Deployment Diagram



➤ Conclusion

The Deployment Diagram provides a clear visualization of the physical infrastructure required for the Ambulance Dispatching System. It shows how client devices (dispatchers, ambulance crew, and hospitals) interact with backend servers and databases.

By mapping software to hardware nodes, the deployment diagram ensures that the system is robust, scalable, and capable of handling real-time emergency response operations efficiently.



Software Engineering

A D Patel Institute of
Technology



AY:2025-26
SEM-V, Div: IT1

Practical -10

Aim: Design the Test Cases for software domain problem.

➤ Test Cases for Ambulance Dispatching System

❖ Black Box Testing

Definition: Black Box Testing is a functional testing technique where the tester evaluates the system without knowing its internal code or structure. Test cases are designed based on requirements, inputs, and expected outputs.

Purpose:

- Verify that the system meets functional requirements.
- Check system behavior under different scenarios.
- Ensure correct outputs for given inputs.

Example Use in System: Logging incidents, sending dispatch alerts, generating reports, updating ambulance status.

❖ White Box Testing

Definition: White Box Testing is a structural testing technique where the tester examines the internal logic, code paths, and data flow of the application.

Purpose:

- Ensure all code paths are tested.
- Detect logical errors, loops, and conditions.
- Validate that internal components work correctly.

Example Use in System: Code for ambulance assignment, status update algorithms, timestamp logging, and database queries.

❖ Black Box Test Cases

Test Case ID	Test Scenario	Test Steps	Input Data	Expected Output	Pass/Fail	Remarks
BB-01	Log a new incident	1. Open dispatcher portal 2. Enter patient and caller details 3. Submit	Patient Name: John Doe, Location: Main St, Priority: High	Incident successfully logged, ID generated	Pass/Fail	Verify correct ID assigned
BB-02	Assign ambulance	1. Select an available ambulance 2. Confirm assignment	Ambulance ID: A12, Incident ID: I23	Ambulance status updated to 'Assigned', crew receives alert	Pass/Fail	Verify notification sent
BB-03	Update ambulance status	1. Ambulance crew opens mobile app 2. Update status to 'En route'	Status: En route	Status reflected in dispatcher portal in real-time	Pass/Fail	Test real-time update

 Software Engineering	A D Patel Institute of Technology	 AY:2025-26 SEM-V, Div: IT1
--	--	--

BB-04	Transfer patient info to hospital	1. Dispatcher submits patient info 2. Hospital staff confirms	Patient ID: P101, Hospital ID: H07	Hospital portal receives patient details and confirms	Pass/Fail	Verify hospital receives correct data
BB-05	Generate incident report	1. Dispatcher selects completed incident 2. Click 'Generate Report'	Incident ID: I23	PDF report generated with complete details	Pass/Fail	Verify report content accuracy

❖ White Box Test Cases

Test Case ID	Test Scenario	Test Steps	Input Data	Expected Output	Pass/Fail	Remarks
WB-01	Incident logging function	1. Call logIncident() method with valid data	Patient details, Caller details	New incident record created in DB	Pass/Fail	Verify DB insertion logic
WB-02	Ambulance assignment algorithm	1. Call assignAmbulance() with available ambulances	Ambulance List, Incident ID	Returns closest available ambulance	Pass/Fail	Check selection logic correctness
WB-03	Status update function	1. Call updateStatus() with 'On scene' status	Ambulance ID, Status='On scene'	Status updated in DB, timestamp recorded	Pass/Fail	Verify timestamp logging
WB-04	Patient info transfer	1. Call transferPatientInfo() to hospital API	Patient ID, Hospital ID	API returns success response	Pass/Fail	Test API integration logic
WB-05	Report generation function	1. Call generateReport() for a completed incident	Incident ID	PDF report generated with correct fields	Pass/Fail	Verify report generation logic

❖ Conclusion

The test cases for the Ambulance Dispatching System demonstrate a comprehensive approach to quality assurance. Black Box Testing ensures that the system meets all functional requirements from the user's perspective, verifying inputs, outputs, and system behavior under various scenarios. White Box Testing complements this by examining the internal logic, code paths, and data flow, ensuring that the system's implementation is correct and reliable.

By combining both testing strategies, the system is thoroughly validated for accuracy, reliability, and real-time performance, reducing the risk of errors during actual operation. This structured approach provides a clear framework for developers and testers, ensuring that the dispatching system is robust, user-friendly, and capable of handling real-world emergency situations efficiently.



Software Engineering

A D Patel Institute of
Technology



AY:2025-26
SEM-V, Div: IT1

Practical -11

Aim: Generate SRS Document as per given format.

Software Requirements Specification (SRS)

For: Ambulance Dispatching System (ADS)

Version: 1.0 **Date:** September 11, 2025

Author: [Rishi Patel, Mayur Bharwad, Krut Patel]

1. Introduction

1.1 Purpose

This document provides a detailed specification for the Ambulance Dispatching System (ADS). It describes the system's functional and non-functional requirements based on the provided Entity-Relationship diagram. This SRS is intended for project managers, developers, and testers to build and verify the system correctly.

1.2 Scope

The ADS is a critical software system designed to manage and coordinate emergency medical response. The system will cover the entire lifecycle of an emergency incident, from the initial call to patient admission. The system's scope includes managing dispatchers, ambulances, patients, hospitals, and the dispatching process itself.

1.3 Definitions, Acronyms, and Abbreviations

- **ADS:** Ambulance Dispatching System
- **ERD:** Entity-Relationship Diagram
- **PK:** Primary Key
- **FK:** Foreign Key

2. Overall Description

2.1 Product Perspective

The ADS is a database-driven application that will replace manual or legacy dispatching systems. It is designed to improve response times and resource management. It operates based on the entities and relationships defined in the project's ERD.

2.2 User Characteristics

- **Dispatchers:** Trained professionals who receive emergency calls, enter incident data using the system, and dispatch ambulances.
- **Ambulance Crew (EMTs/Paramedics):** Field personnel who receive dispatch information and update their status.
- **System Administrators:** Responsible for system maintenance, user management, and data integrity.

2.3 Constraints

- The system must accurately reflect the data structure defined in the ERD.
- All data must be stored in a relational database.



Software Engineering

A D Patel Institute of
Technology



AY:2025-26
SEM-V, Div: IT1

- The system must ensure referential integrity using primary and foreign keys.

3. Specific Requirements

This section details the specific requirements derived from the provided ER diagram.

3.1 Functional Requirements

FR-001: Incident Call Management

- The system shall allow a dispatcher to create and manage an Incident Call record.
- Each call record must capture the caller_name , mobile_no , incident Location , incident_type , and Priority_level.
- The system must link each Incident Call to the Dispatcher who initiated it and the relevant Patient.

FR-002: Dispatcher Management

- The system must be able to add, update, and view Dispatcher profiles.
- Each dispatcher profile must include their Name, mobile_no, Shift-time, and a count of cases_assigned.

FR-003: Ambulance Fleet Management

- The system shall maintain a database of all ambulances in the fleet.
- Each ambulance record must contain its Vehicle_no , Ambulance_type , capacity , assigned Driver_name , and Driver_mobile_no.
- The system must track the real-time Status and Location of each ambulance.

FR-004: Patient Record Management

- The system must allow for the creation and management of Patient records.
- A patient record must include the Patient_name , Date_of_birth, mobile_no, and emergency_level.

FR-005: Hospital Directory

- The system shall maintain a directory of Hospital entities.
- Each hospital record must include the Hospital_name, Location, contact_no, patient Capacity, and available Emergency_services.

FR-006: Dispatching Process

- The system must have a feature to log a dispatch event.
- A dispatch record must link one Dispatcher, one Ambulance, and the corresponding Incident Call using foreign keys.
- The system must facilitate the process of admitting an ambulance to a hospital.

3.2 Non-Functional Requirements

NFR-001: Data Integrity

- The system must enforce all primary and foreign key constraints as defined in the ERD. An Incident Call cannot be created without a valid Patient_id. A dispatch record cannot be created without a



Software Engineering

A D Patel Institute of
Technology



AY:2025-26
SEM-V, Div: IT1

valid Ambulance_id and Call_id.

NFR-002: Performance

- Database queries for finding an available ambulance or logging a new incident call must be executed in under 2 seconds to ensure rapid response times.

NFR-003: Reliability

- The system must be available 24/7. It must include data backup and recovery mechanisms to prevent data loss.

NFR-004: Security

- Access to patient data (e.g., Patient_name, Date_of_birth) must be restricted to authorized personnel (Dispatchers, Admins). All sensitive data must be encrypted.

NFR-005: Usability

- The user interface for dispatchers must be highly intuitive, allowing for quick entry of incident call data to minimize delay in dispatching an ambulance.



Software Engineering

A D Patel Institute of
Technology



AY:2025-26
SEM-V, Div: IT1

Practical -12

Aim: Why Agile process models with DevOps are recommended in big companies like Infosys. Justify with one of the case studies (You can explore it with any tool like JIRA). Write a report of 3-5 pages on case study.

➤ Agile Process

Agile process models combined with DevOps are highly recommended for large companies like Infosys because they enable the organization to overcome the challenges of scale and complexity inherent in large-scale software projects. By adopting an integrated approach, these companies can achieve faster time-to-market, improved product quality, enhanced collaboration, and greater business value.

➤ Agile and DevOps in Large Enterprises

Large, global companies like Infosys operate on an immense scale, managing thousands of projects for clients across various industries. Traditional project management methodologies, such as the Waterfall model, often lead to long development cycles, delayed feedback, and a high risk of project failure due to a lack of flexibility. Agile and DevOps, when used together, create a powerful framework to address these issues.

Agile focuses on delivering value through small, iterative cycles (sprints). It emphasizes:

- **Customer collaboration** over contract negotiation.
- **Responding to change** over following a rigid plan.
- **Working software** over comprehensive documentation.
- **Individuals and interactions** over processes and tools.

This approach breaks down large projects into manageable chunks, allowing teams to deliver new features frequently and incorporate customer feedback early and often.

➤ **DevOps** is a cultural, automation, and technical practice that unifies development (Dev) and operations (Ops) teams. It extends the principles of Agile beyond the development phase by automating the software delivery pipeline. Key practices include:

- **Continuous Integration (CI):** Regularly merging code changes into a central repository and automatically testing the build.



Software Engineering

A D Patel Institute of
Technology



AY:2025-26
SEM-V, Div: IT1

- - **Continuous Delivery/Deployment (CD):** Automatically building, testing, and deploying the application to production.
 -
 - **Infrastructure as Code (IaC):** Managing and provisioning infrastructure using code, rather than manual processes.
- **The Synergy:** Agile and DevOps are complementary. Agile provides the "what" and "why" (customer-centric, iterative development), while DevOps provides the "how" (automation, collaboration, and continuous flow) to make it happen at scale. For a company like Infosys, this combined approach enables them to deliver solutions faster, with higher quality, and with less friction between teams.
- **Case Study: DevOps Transformation at a Global Financial Services Company**
This case study, based on a real-world scenario where Infosys assisted a large American financial services company, highlights the transformative power of a combined Agile-DevOps approach. The client, a multinational organization with a vast IT portfolio, faced several critical challenges typical of large enterprises:
1. **Slow Time-to-Market:** The product delivery and release process was manual and inefficient, leading to bottlenecks and an inability to keep pace with the rapidly changing financial technology landscape.
 2. **Poor Quality and High Risk:** Delayed feedback loops, a lack of automated code quality checks, and manual deployment processes resulted in frequent errors and application downtime during production releases.
 3. **Siloed Teams:** The development and operations teams worked in isolation, leading to communication gaps, finger-pointing, and a lack of shared responsibility.
 4. **Inefficient Processes:** Tedious manual tasks, from code integration to post-deployment validation, consumed significant man-hours and reduced overall team productivity.

➤ **The Agile-DevOps Solution**



Software Engineering

A D Patel Institute of
Technology



AY:2025-26
SEM-V, Div: IT1

Infosys partnered with the client to implement a comprehensive Agile-DevOps transformation. The core of the solution involved a cultural shift, process re-engineering, and the adoption of an integrated toolchain.

Cultural and Process Transformation

- **Agile Mindset:** The company moved from a rigid, project-centric model to a product-centric one, where small, cross-functional teams worked in short, two-week sprints. Each team had a shared product backlog and was responsible for the entire software lifecycle, from development to production.
- **DevOps Culture:** The traditional silos between development, QA, and operations were broken down. A culture of **shared responsibility and ownership** was fostered, where teams were encouraged to "build, fix, and operate together."

➤ Toolchain Implementation with JIRA and Automation

A unified toolchain was implemented to automate the software delivery pipeline, with **Jira** as the central hub for project management and collaboration.

- **Project Management in Jira:** All user stories, epics, and tasks were managed in a single Jira instance. Teams used **Jira's Kanban and Scrum boards** to visualize their workflow, track progress in real-time, and hold daily stand-ups to discuss blockers. The backlog was prioritized collaboratively by the product owner, ensuring teams always worked on the most valuable features.
- **Continuous Integration (CI):** Developers committed their code to a central Git repository. A CI tool, such as **Jenkins**, was integrated with Jira to automatically trigger a build and run unit tests for every code commit. This automation provided **immediate feedback** on code quality and prevented broken builds from reaching later stages.
- **Continuous Delivery/Deployment (CD):** The successful builds were automatically packaged and deployed to a staging environment. The team leveraged **Jira Automation** to create and update tickets for each deployment, providing a real-time audit trail. A **blue-green deployment strategy** was introduced for production releases, enabling zero-downtime updates and a quick rollback if issues arose.
- **Automated Quality Checks:** The CI/CD pipeline integrated automated testing tools



Software Engineering

A D Patel Institute of
Technology



AY:2025-26
SEM-V, Div: IT1

(**Selenium** for functional tests) and static code analysis tools (**SonarQube**). When a code quality issue was found, the pipeline would automatically fail and create a new **bug ticket in Jira**, which was then assigned back to the development team for immediate resolution. This drastically reduced the number of defects reaching production.

- **Real-time Monitoring:** Post-deployment, tools like **Grafana** and **Prometheus** were used to monitor application performance and system health. These tools were integrated with Jira and **Opsgenie** to automatically create tickets and alert the responsible team members in case of an incident. This allowed for rapid issue resolution, reducing the Mean Time to Recover (MTTR) from hours to minutes.

➤ Outcomes and Business Justification

The Agile-DevOps transformation yielded significant and quantifiable results for the financial services company, justifying the investment.

- **Increased Release Frequency:** The company moved from a bi-monthly release cycle to monthly, and in some cases, weekly releases. This 40% increase in release frequency allowed the business to respond to market demands and launch new features much faster than competitors.
- **Reduced Development and Operations Costs:** The overall effort for the software delivery lifecycle was reduced by approximately **70%** through extensive automation. This freed up developers and operations engineers to focus on innovation rather than manual, repetitive tasks.
- **Improved Product Quality:** The automation of code quality checks and continuous testing led to a **70% increase in code quality**. The number of production incidents and application downtime were significantly reduced, leading to higher system stability.
- **Enhanced Team Productivity and Collaboration:** Streamlined release processes and transparent workflows in Jira led to a **25% increase in team productivity**. The cultural shift towards collaboration and shared ownership fostered a more engaged and innovative workforce.
- **Higher Customer Satisfaction:** With more frequent, high-quality releases and minimal downtime, the end-user experience improved dramatically, leading to higher customer satisfaction and retention.



Software Engineering

A D Patel Institute of
Technology



AY:2025-26
SEM-V, Div: IT1

➤ Conclusion

In conclusion, for a large company like Infosys, Agile and DevOps are not just buzzwords; they are a strategic imperative. The case study demonstrates that by combining the principles of iterative delivery with the power of automation and an integrated toolchain like Jira, an organization can achieve a complete business and cultural transformation that results in faster delivery, better quality, and a sustainable competitive advantage.